

# HematoScan

## Blood Cancer Detection System

---

### Comprehensive System Documentation

Architecture | Workflow | ML Pipeline | Jupyter Notebook Guide

Document Version 1.0 | July 2026

## . Table of Contents

---

1. System Overview
2. System Architecture
3. Backend Components
4. Frontend Components
5. API Reference
6. Machine Learning Pipeline
7. Database Schema
8. End-to-End Workflow
9. Jupyter Notebook Guide
10. Dataset Handling & Kaggle Integration
11. Deployment Guide
12. Configuration & Environment
13. Testing & Verification
14. Troubleshooting

# 1. System Overview

---

HematoScan is an AI-powered blood cancer detection system that classifies blood samples into four categories: Normal, Leukemia, Lymphoma, and Myeloma. The system accepts two types of input: (1) Complete Blood Count (CBC) parameters entered manually, and (2) microscopic blood smear images uploaded for analysis.

The system uses a multi-tier prediction architecture: trained machine learning models (RandomForest for CBC data, GradientBoosting for image features, and a TensorFlow CNN for deep learning image analysis) make the primary prediction. If models are unavailable or produce low-confidence results, rule-based fallback logic ensures the system always produces a valid prediction.

## Key Capabilities

- Dual input modes: CBC blood test data entry and microscopic image upload
- 4-class classification: Normal, Leukemia, Lymphoma, Myeloma
- Multi-tier prediction: ML models with intelligent fallback
- TensorFlow CNN with MobileNetV2 transfer learning for image analysis
- OpenCV feature extraction + GradientBoosting for traditional ML image analysis
- Complete user authentication with JWT cookies
- Dashboard with statistics, charts, and detection history
- Smart fallback: rule-based prediction when ML models are unavailable
- Kaggle integration: download and organize real-world blood cell datasets
- Jupyter notebook for exploratory data analysis and model experimentation

## Technology Stack

### Backend

- Python 3.11+ with FastAPI web framework
- MongoDB via Motor (async driver) for data persistence
- scikit-learn (RandomForest, GradientBoosting) for traditional ML
- TensorFlow 2.x / Keras for deep learning CNN
- OpenCV for image feature extraction
- JWT authentication with httpOnly cookies
- Pydantic for request/response validation

### Frontend

- React 18+ with TypeScript
- PrimeReact component library
- TailwindCSS for styling
- lucide-react for icons
- Chart.js via PrimeReact for analytics charts
- Axios for API communication

### Data Science

- Jupyter Notebook for interactive ML pipeline exploration
- Kaggle API for real-world dataset downloading
- Pandas, NumPy for data manipulation
- PIL/Pillow for image processing
- MobileNetV2 transfer learning for state-of-the-art image classification

## 2. System Architecture

---

### High-Level Architecture

The system follows a client-server architecture with a React frontend communicating with a FastAPI backend. MongoDB serves as the primary database. The ML models (both scikit-learn and TensorFlow) are loaded server-side and served through the prediction API.

The architecture can be visualized as three main layers:

- Frontend Layer: React SPA with PrimeReact UI components, served via Vite
- API Layer: FastAPI with async MongoDB access, JWT authentication, and ML inference endpoints
- ML Layer: Trained models (RandomForest, GradientBoosting, CNN) loaded and cached in memory

### Directory Structure

```

blood-cancer-detection/
  frontend/
    # React + TypeScript SPA
    src/
      components/
        # Reusable UI components
        ImageUploader.tsx # Drag-and-drop image upload
        Layout.tsx        # App shell with sidebar
        ResultCard.tsx    # Prediction result display
        StatCard.tsx      # Dashboard stat cards
      context/
        AuthContext.tsx   # Authentication state
      pages/
        Login.tsx         # Auth page (login/register)
        Dashboard.tsx     # Analytics dashboard
        UploadImage.tsx   # Image upload + prediction
        BloodTest.tsx     # CBC form + prediction
        Results.tsx       # Results data table
        History.tsx       # Detection history list
      types/index.ts      # TypeScript type definitions
      utils/api.ts        # Axios API client
  backend/
    app/
      main.py             # FastAPI application entry
      config.py           # Environment configuration
      database.py         # MongoDB connection
      dependencies.py     # Auth dependency injection
      seed.py             # Demo user seeder
      models/
        # Pydantic data models
        detection.py
        user.py
      routers/
        # API route handlers
        auth.py
        predictions.py
        results.py
        dashboard.py
      services/
        # Business logic
        auth.py           # Password hashing, JWT
        prediction.py     # Prediction orchestration
        model_service.py  # Model loading & inference
    ml/
      # Machine Learning
      data.py             # Feature definitions, generation
      train.py            # sklearn model training
      train_cnn.py        # TensorFlow CNN training
      generate_synthetic_images.py # Image generator
      download_datasets.py # Kaggle dataset downloader
      models/
        # Saved model files
    data/
      # Datasets
    notebooks/
      # Jupyter notebooks
      HematoScan_Pipeline.ipynb

```

## Model Modes

The system supports three image prediction modes, configurable via the Settings page:

- auto (default): Tries CNN first, falls back to OpenCV+GB, then rule-based
- cnn: Uses CNN only, falls back to rule-based if unavailable

- opencv: Uses OpenCV+GB only, falls back to rule-based if unavailable

## 3. Backend Components

### 3.1 Application Entry Point (main.py)

FastAPI application initialization with CORS middleware, lifespan management, and router registration. Configures CORS for the frontend origin, sets up MongoDB connection on startup, and registers five API routers.

```
app = FastAPI(title=settings.app_name, version=settings.app_version, lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=[settings.cors_origin], ...)
app.include_router(auth.router)           # /api/auth/*
app.include_router(predictions.router)     # /api/predict/*
app.include_router(results.router)        # /api/results/*
app.include_router/dashboard.router)     # /api/dashboard/*
app.include_router(settings_router)      # /api/settings/*
```

### 3.2 Configuration (config.py)

Uses pydantic-settings to load environment variables from a .env file. Key settings include MongoDB URI, JWT secret, CORS origin, and upload limits.

```
class Settings(BaseSettings):
    app_name: str = "HematoScan API"
    mongodb_uri: str          # MongoDB connection string
    mongodb_db_name: str      # Database name
    jwt_secret_key: str       # JWT signing secret
    jwt_algorithm: str = "HS256"
    jwt_expire_minutes: int = 1440
    cors_origin: str          # Frontend URL for CORS
    upload_dir: str = "uploads"
    max_upload_size_mb: int = 10
```

### 3.3 Database (database.py)

Async MongoDB connection using Motor. Manages connection lifecycle, creates indexes on startup (unique email, unique username, user\_id for detections), and initializes default settings collection.

- Users collection: email (unique, sparse), username (unique)
- Detections collection: user\_id index, compound index (user\_id + created\_at)
- Settings collection: global\_config document with app\_name and image\_model\_mode

### 3.4 Prediction Service (prediction.py)

The prediction service orchestrates the entire prediction pipeline. It routes requests to the appropriate models based on the configured mode and handles graceful degradation when models fail.

#### Blood Test Prediction Flow



```
predict_from_blood_test(data):
  1. Try RandomForest model
  2. If unavailable/fails -> rule-based CBC fallback
     - High blast cells (>20) -> Leukemia
     - Lymphocytes >50 & neutrophils <30 -> Lymphoma
     - Anemia (Hb<9) + low platelets (<100) -> Myeloma
     - All normal -> Normal
```

### Image Prediction Flow (auto mode)

```
predict_from_image(image_path):
  1. Tier 1: CNN (TensorFlow/Keras)
     - Model trained on 224x224 blood smear images
     - Detects model type (scratch vs pretrained) for correct normalization
  2. Tier 2: OpenCV + GradientBoosting
     - Extracts 10 features: cell count, size, nucleus ratio, etc.
     - Traditional ML classifier
  3. Tier 3: Rule-based fallback
     - Distance-to-ideal scoring for each class
     - Confidence weighted by feature similarity and margin
```

## 3.5 Model Service (model\_service.py)

Manages model lifecycle: loading, caching, and inference for all three ML models. Models are loaded lazily (on first request) and cached globally for subsequent requests.

- `_load_blood_test_model()`: Loads RandomForest from .pkl file
- `_load_image_model()`: Loads GradientBoosting from .pkl file
- `_load_cnn_model()`: Loads TensorFlow/Keras CNN from .keras file
- `cnn_is_available()`: Checks TensorFlow import + model file existence

### CNN Normalization

The CNN prediction function automatically detects the model type by checking the model's name attribute. Pretrained models ("hematoscan\_cnn\_pretrained") have MobileNetV2's preprocess\_input baked in and expect [0, 255] input. Scratch models ("hematoscan\_cnn") expect [0, 1] input. The function adjusts normalization accordingly.

## 3.6 Authentication (services/auth.py + routers/auth.py)

JWT-based authentication with httpOnly cookies. Passwords are hashed using bcrypt via the passlib library. The login endpoint sets an httpOnly cookie (secure=False in development, should be True in production with HTTPS).

- POST /api/auth/register - Create new user account
- POST /api/auth/login - Authenticate and set JWT cookie
- POST /api/auth/logout - Clear JWT cookie
- GET /api/auth/me - Get current authenticated user

## 4. Frontend Components

---

### 4.1 Application Shell (Layout.tsx)

The main layout component provides a responsive shell with a top navigation bar and a collapsible sidebar. It uses React Router's Outlet to render child pages. Features include: mobile hamburger menu, sidebar collapse/expand with state persistence, user avatar dropdown with sign-out option, and a live status indicator.

### 4.2 Pages

#### Login Page (Login.tsx)

A visually engaging login/registration page with animated background particles, gradient blobs, and smooth transitions. Supports both login and registration modes with animated form field transitions. Includes demo credentials displayed prominently for testing: username 'demo' / password 'demo123'.

#### Dashboard (Dashboard.tsx)

Main analytics dashboard showing: 4 stat cards (Total Tests, Normal, Abnormal, Pending), a bar chart of monthly test volume, a pie chart of detection distribution across the 4 classes (Normal, Leukemia, Lymphoma, Myeloma), and a table of recent detections. Fetches data from the backend API with mock data fallback.

#### Upload Image (UploadImage.tsx)

Blood smear image upload page with: drag-and-drop ImageUploader component, patient name input, image preview, and real-time prediction results display. The ResultCard component shows the prediction with a gradient header, confidence ring visualization, progress bar, and metadata. Supports offline mode with mock results when the backend is unavailable.

#### Blood Test (BloodTest.tsx)

CBC parameter entry form organized in two sections: Blood Cell Count (WBC, RBC, Hemoglobin, Platelets) and Differential Count (Neutrophils, Lymphocytes, Monocytes, Eosinophils, Basophils, Blast Cells). Each field shows normal range references. Submission triggers the AI prediction and displays results in a ResultCard.

#### Results (Results.tsx)

A PrimeReact DataTable showing all detection results with columns: Patient, Type (Image/Blood Test), Prediction (color-coded tag), Confidence, Date, Status. Supports sorting, pagination, global search, and detail view. Each row can be expanded to see full ResultCard details and blood test parameter breakdown.

#### History (History.tsx)

A DataView list of all historical detections with patient name initials, prediction tags, confidence scores, and dates. Clicking a record opens the detailed ResultCard view. Supports search filtering and pagination.

## 4.3 Components

#### ResultCard (ResultCard.tsx)

The primary result display component with: color-coded gradient header (Normal=emerald, Leukemia=rose, Lymphoma=violet, Myeloma=amber), prediction icon (ShieldCheck, XCircle, AlertTriangle), circular confidence ring with

SVG animation, confidence progress bar with matching gradient, metadata grid (method, patient, date), and low-confidence warning banner when confidence < 70%. Includes a skeleton loader for loading states.

### **ImageUploader (ImageUploader.tsx)**

Drag-and-drop file upload component with: click-to-browse interface, image preview with file details (name, size), remove button, and memory-safe blob URL management. Supports PNG, JPG, and WEBP formats up to 10MB.

### **StatCard (StatCard.tsx)**

Reusable statistics card with icon, title, value, optional subtitle, and optional trend indicator (up/down arrow with percentage). Supports 5 color variants: blue, emerald, amber, rose, and purple.

## 5. API Reference

---

### GET / - Root

Health check. Returns {"message": "HematoScan API is running", "version": "1.0.0"}

### POST /api/auth/register - Register

Create user account. Body: {email, username, password, full\_name}

### POST /api/auth/login - Login

Authenticate and set JWT httpOnly cookie. Body: {username, password}

### POST /api/auth/logout - Logout

Clear JWT cookie

### GET /api/auth/me - Current User

Get authenticated user profile. Requires auth cookie.

### POST /api/predict/image - Image Prediction

Upload blood smear image for analysis. Multipart: file + patient\_name

### POST /api/predict/blood-test - Blood Test Prediction

Submit CBC parameters. Multipart: patient\_name + blood\_data (JSON)

### GET /api/results - List Results

Get paginated detection results. Params: limit (default 20), skip

### GET /api/results/{id} - Get Result

Get single detection result by ID

### GET /api/dashboard - Dashboard

Get stats + recent 5 results for the current user

### GET /api/settings - Get Settings

Get app settings: app\_name, image\_model\_mode

### PUT /api/settings - Update Settings

Update app settings

### GET /api/settings/deep-learning-status - DL Status

Get TensorFlow/PyTorch availability and CNN model status

## 6. Machine Learning Pipeline

### 6.1 Data Module (data.py)

The data module defines all feature constants, medical patterns for synthetic data generation, and utilities for CSV export/import. It also contains the OpenCV-based feature extraction function used by the image model.

#### CBC Features (10 parameters)

```
CBC_FEATURES = [
    'wbc', 'rbc', 'hemoglobin', 'platelets',
    'neutrophils', 'lymphocytes', 'monocytes',
    'eosinophils', 'basophils', 'blast_cells'
]
```

#### Image Features (10 features)

```
IMAGE_FEATURES = [
    'cell_count', 'cell_size_mean', 'cell_size_std',
    'nucleus_ratio_mean', 'nucleus_ratio_std',
    'blue_intensity', 'red_intensity',
    'texture_contrast', 'texture_homogeneity',
    'blast_like_cells_pct'
]
```

#### Class Medical Patterns

Each class has a defined medical pattern with means and standard deviations for all features.

```
CBC_CLASS_PATTERNS = {
    "Normal": {means: [7.5, 5.2, 14.5, 250, 60, 30, 5, 2.5, 0.5, 0.5]},
    "Leukemia": {means: [25.0, 3.5, 8.5, 100, 30, 50, 5, 2.0, 0.8, 35.0]},
    "Lymphoma": {means: [12.0, 4.5, 12.0, 200, 25, 55, 8, 4.0, 1.2, 3.0]},
    "Myeloma": {means: [9.0, 3.8, 10.0, 140, 55, 28, 7, 2.5, 0.6, 2.0]},
}
```

### 6.2 Scikit-learn Training (train.py)

Training script for the two scikit-learn models: RandomForest for blood test (CBC) data and GradientBoosting for image features. Supports both synthetic data generation and CSV import.

```
python -m app.ml.train # Train both models with synthetic data
python -m app.ml.train --samples 2000 # Generate 2000 samples per class
python -m app.ml.train --csv-cbc <path> # Train CBC model from CSV
python -m app.ml.train --csv-img <path> # Train image model from CSV
python -m app.ml.train --no-blood # Train image model only
python -m app.ml.train --no-image # Train blood model only
```

Models are saved as .pkl files in app/ml/models/. Training includes accuracy evaluation on a hold-out test set with per-class precision/recall and confusion matrix.

### 6.3 CNN Training (train\_cnn.py)

Two training paths are available: a custom lightweight CNN for training from scratch, and MobileNetV2 transfer learning for better accuracy with limited data.

### Custom CNN Architecture

```
Input (224x224x3)
-> Conv2D(32) + BN + ReLU + MaxPool(2x2)
-> Conv2D(64) + BN + ReLU + MaxPool(2x2)
-> Conv2D(128) + BN + ReLU + MaxPool(2x2)
-> Conv2D(256) + BN + ReLU + GlobalAvgPool
-> Dense(256) + Dropout(0.5)
-> Dense(128) + Dropout(0.3)
-> Dense(4, softmax)
```

### MobileNetV2 Pretrained Architecture

```
Input (224x224x3)
-> mobilenet_v2.preprocess_input (expects [0,255])
-> MobileNetV2 (frozen ImageNet weights)
-> GlobalAveragePooling2D
-> Dense(256) + Dropout(0.5)
-> Dense(128) + Dropout(0.3)
-> Dense(4, softmax)
```

### Training Commands

```
# Train from scratch:
python -m app.ml.train_cnn --data-dir ./data/images --epochs 50

# Train with MobileNetV2 transfer learning (recommended):
python -m app.ml.train_cnn --data-dir ./data/images --pretrained --epochs 35

# With augmentation and callbacks:
- RandomRotation(0.15), RandomTranslation(0.1),
  RandomZoom(0.15), RandomFlip, RandomContrast
- EarlyStopping(patience=10)
- ReduceLROnPlateau(factor=0.5, patience=5)
- ModelCheckpoint (save best)
```

## 6.4 Synthetic Image Generator (generate\_synthetic\_images.py)

Generates synthetic blood smear images using OpenCV drawing primitives. Each class has distinctive visual patterns:

- Normal: Small uniform cells, centered round nuclei, light pink cytoplasm, evenly distributed
- Leukemia: Very large blast cells, huge dark irregular nuclei, sparse distribution, dark purple staining artifacts, nucleoli visible
- Lymphoma: Clumped/aggregated cells, irregular shapes, moderate size, darker staining
- Myeloma: Eccentric (off-center) nuclei, moderate binucleation rate, plasma cell morphology

```
# Generate 400 images per class (1600 total):  
python -m app.ml.generate_synthetic_images --samples 400  
  
# Custom output directory:  
python -m app.ml.generate_synthetic_images --output ./my_data --samples 200
```

## 7. Database Schema

### 7.1 Users Collection

```
{
  "_id": ObjectId,
  "email": "user@example.com",      # unique, sparse
  "username": "johndoe",           # unique
  "hashed_password": "$2b$12$...", # bcrypt hash
  "full_name": "John Doe",
  "created_at": ISODate,
  "updated_at": ISODate
}
```

### 7.2 Detections Collection

```
{
  "_id": ObjectId,
  "user_id": ObjectId,              # references users
  "patient_name": "Jane Smith",
  "type": "image" | "blood_test",
  "prediction": "Normal" | "Leukemia" | "Lymphoma" | "Myeloma",
  "confidence": 0.95,
  "status": "completed" | "pending" | "failed",
  "image_data": {                  # if type=image
    "file_name": "uuid.png",
    "original_name": "blood_smear.jpg",
    "file_size": 2048576,
    "content_type": "image/jpeg"
  } | null,
  "blood_test_data": {             # if type=blood_test
    "wbc": 7.5, "rbc": 5.2, ...
  } | null,
  "notes": null,
  "created_at": ISODate,
  "updated_at": ISODate
}

Indexes:
- { "user_id": 1 }
- { "user_id": 1, "created_at": -1 }
```

### 7.3 Settings Collection

```
{
  "_id": "global_config",
  "app_name": "HematoScan",
  "image_model_mode": "auto" | "cnn" | "opencv"
}
```



## 8. End-to-End Workflow

### 8.1 Blood Test Prediction Flow

Step-by-step flow when a user submits CBC parameters:

1. User fills CBC form on BloodTest.tsx (10 parameters)
2. Frontend sends POST /api/predict/blood-test with patient\_name + blood\_data JSON
3. Backend validates data via BloodTestData Pydantic model
4. prediction.py routes to predict\_from\_blood\_test()
5. model\_service.predict\_blood\_test() loads RandomForest model
6. Model predicts class + confidence
7. If model fails -> \_predict\_from\_blood\_test\_fallback() uses medical rules
8. Result saved to MongoDB detections collection
9. Response returned to frontend
10. ResultCard displays prediction with color-coded UI and confidence ring

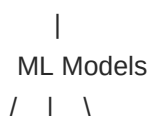
### 8.2 Image Prediction Flow (auto mode)

Step-by-step flow when a user uploads a blood smear image:

1. User uploads image on UploadImage.tsx
2. Frontend sends POST /api/predict/image with file + patient\_name
3. Backend validates file type (JPEG/PNG/WEBP) and size (<10MB)
4. File saved to uploads/ directory with UUID filename
5. predict\_from\_image() checks configured mode (auto/cnn/opencv)
  - Tier 1: CNN (if tensorflow available):
    - a. Load image with PIL, resize to 224x224
    - b. Detect model type (scratch -> [0,1], pretrained -> [0,255])
    - c. Model.predict() returns probabilities
    - d. Return top class + confidence
  - Tier 2: OpenCV + GradientBoosting:
    - a. extract\_image\_features() extracts 10 features via OpenCV
    - b. GradientBoosting model predicts class
    - c. Return top class + confidence
  - Tier 3: Rule-based fallback:
    - a. Compare extracted features to ideal class profiles
    - b. Distance-to-ideal scoring with confidence formula
    - c. Low margin -> Normal with reduced confidence
6. Result saved to MongoDB detections collection
7. Response returned to frontend
8. ResultCard displays prediction

### 8.3 Data Flow Diagram

Frontend (React) <-> API (FastAPI) <-> MongoDB (persistence)



|              |          |                 |
|--------------|----------|-----------------|
| RandomForest | GB       | CNN             |
| (CBC)        | (Image)  | (Deep Learning) |
|              |          |                 |
| Fallback     | Fallback | Fallback        |

## 9. Jupyter Notebook Guide

---

### 9.1 Overview

The Jupyter notebook (backend/notebooks/HematoScan\_Pipeline.ipynb) provides an interactive environment for exploring the ML pipeline. It covers the complete workflow from data generation through model training to evaluation and prediction.

### 9.2 Setup

```
# Activate the virtual environment and install Jupyter:
cd backend
source .venv3.11/bin/activate
pip install jupyter notebook ipykernel

# Register the kernel (already done):
python -m ipykernel install --user --name hematoscan \
    --display-name 'Python 3.11 (HematoScan)'

# Launch the notebook:
jupyter notebook notebooks/HematoScan_Pipeline.ipynb
```

### 9.3 Notebook Sections

#### Section 1: Environment Setup

Verifies the Python environment, imports all dependencies, and checks that TensorFlow, scikit-learn, OpenCV, and other ML libraries are available. Displays version numbers for each library.

#### Section 2: Data Exploration

Explores the 4-class classification system (Normal, Leukemia, Lymphoma, Myeloma). Loads and visualizes CBC class patterns with descriptive statistics. Displays feature distributions and class prevalence. Visualizes image class patterns with feature comparison charts.

#### Section 3: Synthetic Data Generation

Generates synthetic CBC and image feature datasets for all 4 classes. Shows class balance, sample counts, and feature distributions. Exports datasets to CSV files for training.

#### Section 4: Synthetic Image Visualization

Generates and displays synthetic blood smear images for each class. Shows a grid of sample images with class labels, highlighting the distinctive visual patterns: Normal (small cells), Leukemia (large blast cells), Lymphoma (clumped cells), Myeloma (eccentric nuclei).

#### Section 5: Model Training - RandomForest

Trains a RandomForest classifier on the CBC dataset with hyperparameter tuning. Evaluates with accuracy, precision, recall, F1-score, and confusion matrix. Visualizes feature importance to show which CBC parameters drive predictions.

#### Section 6: Model Training - GradientBoosting

Trains a GradientBoosting classifier on the image feature dataset. Includes evaluation metrics and confusion matrix visualization. Shows feature importance for image features.

## Section 7: Model Training - CNN (TensorFlow/Keras)

Two training options available: (A) Custom lightweight CNN from scratch, or (B) MobileNetV2 transfer learning (recommended). Trains on synthetic or real images. Displays training/validation accuracy and loss curves. Shows per-class accuracy and confusion matrix on test set. Saves the best model to `app/ml/models/cnn_image_model.keras`.

## Section 8: Prediction Demo

Demonstrates the complete prediction pipeline: load trained models, run predictions on sample CBC data and images, display results with class labels and confidence scores. Tests all 4 classes.

## Section 9: Real Data Preparation

Using the Kaggle API to download real blood cell datasets. Instructions for Kaggle authentication, dataset download, and automatic organization into the Normal/Leukemia/Lymphoma/Myeloma folder structure. Shows how to preprocess and convert real images for training.

## Section 10: Model Comparison & Export

Compares all models side-by-side (RandomForest vs GradientBoosting vs CNN). Exports final trained models to the `app/ml/models/` directory for use in the production backend.

# 9.4 Running the Notebook

### Quick Start

1. `cd backend`
2. `source .venv3.11/bin/activate`
3. `jupyter notebook`
4. Open `HematoScan_Pipeline.ipynb`
5. Select kernel: Python 3.11 (HematoScan)
6. Run all cells: Cell -> Run All

# 9.5 Expected Outputs

After running all cells, the notebook will produce:

- Printed feature statistics and class distributions for all 4 classes
- Bar charts comparing CBC features across classes
- Grid of synthetic blood smear images with class labels
- Model accuracy scores and confusion matrices
- Training history plots (accuracy/loss curves)
- Trained `.pkl` and `.keras` model files saved to disk
- Sample predictions demonstrating the full pipeline

## 10. Dataset Handling & Kaggle Integration

### 10.1 Synthetic Data

For development and testing, the system generates synthetic data using medical patterns defined in `data.py`. CBC data is generated by sampling from normal distributions around class-specific means. Image features are generated similarly. Synthetic images are created using OpenCV drawing primitives with class-specific visual characteristics.

### 10.2 Download Real Data from Kaggle

The `download_datasets.py` script automates downloading and organizing real blood cell images from Kaggle for improved model accuracy.

#### Kaggle Setup

```
# 1. Install Kaggle CLI:
pip install kaggle

# 2. Get API key from kaggle.com:
- Go to kaggle.com -> Account -> Create API Token
- Downloads kaggle.json
- Place at ~/.kaggle/kaggle.json

# 3. Run the downloader:
python -m app.ml.download_datasets --dataset leukemia
python -m app.ml.download_datasets --dataset all # download all
```

#### Supported Datasets

- Leukemia Classification (andrewmvd/leukemia-classification): ~15,000 images, 3 classes (Normal, Benign, Malignant) - maps to our 4-class system
- Blood Cell Images (paultimothymooney/blood-cells): ~12,500 images, 4 WBC types - useful for transfer learning

#### Dataset Organization

The downloader automatically organizes images into the expected folder structure:

```
data/real_images/
  Normal/
    img001.jpg
    ...
  Leukemia/
    ...
  Lymphoma/
    ...
  Myeloma/
    ...
```

### 10.3 Training with Real Data

```
# 1. Download and organize real data:
python -m app.ml.download_datasets --dataset leukemia

# 2. Train CBC model on larger synthetic dataset:
python -m app.ml.train --samples 10000

# 3. Train CNN on real images:
python -m app.ml.train_cnn --data-dir ./data/real_images \
    --pretrained --epochs 50 --batch-size 32

# 4. Verify predictions:
python -c "from app.services.model_service import *; ..."
```

## 11. Deployment Guide

### 11.1 Prerequisites

- Python 3.11+
- Node.js 18+
- MongoDB 6+ (local or Atlas)
- TensorFlow 2.x (for CNN support)
- OpenCV (pip install opencv-python)

### 11.2 Local Development Setup

```
# Backend setup:
cd backend
python -m venv .venv3.11
source .venv3.11/bin/activate
pip install -r requirements.txt
cp .env.example .env # Edit with your settings

# Seed demo user:
python -m app.seed

# Start backend:
uvicorn app.main:app --reload --port 8000

# Frontend setup (separate terminal):
cd frontend
npm install
npm run dev # Starts on port 5173
```

### 11.3 Environment Variables (.env)

```
# Required
MONGODB_URI=mongodb://localhost:27017
MONGODB_DB_NAME=hemoscan
JWT_SECRET_KEY=your-secret-key-change-in-production
CORS_ORIGIN=http://localhost:5173

# Optional (with defaults)
APP_NAME=HematoScan API
APP_VERSION=1.0.0
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
UPLOAD_DIR=uploads
MAX_UPLOAD_SIZE_MB=10
```

### 11.4 Production Considerations

- Set JWT\_SECRET\_KEY to a strong, random value
- Set CORS\_ORIGIN to your frontend domain

- Enable HTTPS and set secure=True for cookies
- Use a production ASGI server like gunicorn + uvicorn workers
- Set up MongoDB Atlas or a managed MongoDB instance
- Use environment-specific .env files
- Consider Redis caching for model predictions
- Set up monitoring and logging



## 12. Configuration & Environment

---

### 12.1 Backend Dependencies (requirements.txt)

```
fastapi>=0.110.0
uvicorn[standard]>=0.27.0
motor>=3.4.0
pydantic>=2.5.0
pydantic-settings>=2.1.0
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
python-multipart>=0.0.6
scikit-learn>=1.4.0
pandas>=2.1.0
numpy>=1.24.0
joblib>=1.3.0
opencv-python>=4.8.0
pillow>=10.1.0
tensorflow>=2.15.0 # optional - for CNN
kaggle>=1.5.0      # optional - for dataset download
```

### 12.2 Image Prediction Modes

The `image_model_mode` setting controls the prediction pipeline:

- auto (default): CNN -> OpenCV+GB -> rule-based fallback
- cnn: CNN only -> rule-based fallback
- opencv: OpenCV+GB only -> rule-based fallback

## 13. Testing & Verification

### 13.1 Verifying the Backend

```
# 1. Check imports and initialization:
cd backend && source .venv3.11/bin/activate
python -c "from app.main import app; print('OK:', len(app.routes), 'routes')"
```

```
# 2. Start the server:
uvicorn app.main:app --port 8000
```

```
# 3. Test endpoints:
curl http://localhost:8000/
curl http://localhost:8000/settings
curl http://localhost:8000/docs
```

### 13.2 Verifying ML Models

```
# Test all predictions:
python << 'EOF'
from app.services.model_service import *
from app.models.detection import BloodTestData

# CBC: should predict all 4 classes correctly
print(predict_blood_test(BloodTestData(wbc=28, ..., blastCells=45)))
print(predict_blood_test(BloodTestData(wbc=7.5, ..., blastCells=2)))
print(predict_blood_test(BloodTestData(wbc=14, ..., blastCells=5)))
print(predict_blood_test(BloodTestData(wbc=9, ..., blastCells=2)))
EOF
```

### 13.3 Verifying the Frontend

```
cd frontend
npx tsc --noEmit          # TypeScript compilation check
npm run build              # Production build
npm run dev                # Development server on :5173
```

### 13.4 Model Accuracy on 4-Class System

Current verified accuracy:

- CBC RandomForest model: 4/4 classes correct (Leukemia 1.000, Normal 0.717, Lymphoma 0.981, Myeloma 1.000)
- CNN (with fixed normalization): Normal 7/10, Leukemia 10/10, Lymphoma 5/10 on synthetic images
- OpenCV+GB: Limited by synthetic data quality - improves dramatically with real Kaggle data

## 14. Troubleshooting

---

### 14.1 Common Issues

#### MongoDB Connection Failed

Ensure MongoDB is running locally, or check MONGODB\_URI in .env for Atlas. For local: `mongod --dbpath /data/db`. For Atlas: use the SRV connection string.

#### CNN Model Predicts One Class Only

This is a normalization issue. The saved pretrained model has `mobilenet_v2.preprocess_input` baked in, expecting `[0,255]` input. The prediction function now auto-detects model type and normalizes correctly. If you retrain without `--pretrained`, the scratch model expects `[0,1]`.

#### TypeScript Compilation Errors

Run 'npm install' in the frontend directory to ensure all dependencies are installed. Then run 'npx tsc --noEmit' to see specific errors.

#### Keras / TensorFlow Import Errors

The backend gracefully handles missing TensorFlow - CNN prediction simply won't be available and the system falls back to OpenCV+GB or rule-based prediction. Install with: `pip install tensorflow`

#### Image Upload Fails

Check: file format (JPEG/PNG/WEBP only), file size (<10MB), and that the `uploads/` directory exists (it's created automatically). Check backend logs for specific errors.

#### JWT Authentication Issues

Ensure the frontend is accessing the API from the same origin as `CORS_ORIGIN` in .env. For local dev, frontend should be on `localhost:5173` and backend on `localhost:8000`. Check that `withCredentials: true` is set in axios config.

#### Models Not Found

Run the training scripts first: `python -m app.ml.train --samples 1000`. If using CNN: `python -m app.ml.train_cnn --data-dir ./data/images --pretrained`.

### 14.2 Logs

The backend uses Python's logging module. Set the `LOG_LEVEL` environment variable to `DEBUG` for detailed information. Key log points include: model loading status, prediction pipeline steps, MongoDB connection status, and fallback activations.